Name of the module: **FREQ_DOMAIN_CORR**
Location in GIT repository: vspa_common\vspa-lib\freq_domain_corr

Overview:

This module provides frequency domain correction capability for space-time streams including:

- Linear phase ramp correction (common to all space-time streams) corresponding to the fractional timing offset
- Complex gain correction (a separate gain for each space-time stream) corresponding to the amplitude and phase drift

C Functions API:

```
void freq_domain_corr_64sbc( cfloat16_t  *inp_p,
                             cfloat32_t  *gain_cpx_p,
                             cfloat16_t  *scratch_p,
                             int32_t      phase_ramp,
                             int32_t      phase_init,
                             uint32_t     num_streams);

void freq_domain_corr_128sbc( cfloat16_t  *inp_p,
                              cfloat32_t  *gain_cpx_p,
                              cfloat16_t  *scratch_p,
                              int32_t      phase_ramp,
                              int32_t      phase_init,
                              uint32_t     num_streams);

void freq_domain_corr_256sbc( cfloat16_t  *inp_p,
                              cfloat32_t  *gain_cpx_p,
                              cfloat16_t  *scratch_p,
                              int32_t      phase_ramp,
                              int32_t      phase_init,
                              uint32_t     num_streams);

void freq_domain_corr_512sbc( cfloat16_t  *inp_p,
                              cfloat32_t  *gain_cpx_p,
                              cfloat16_t  *scratch_p,
                              int32_t      phase_ramp,
                              int32_t      phase_init,
                              uint32_t     num_streams);

void freq_domain_corr_1024sbc( cfloat16_t  *inp_p,
                               cfloat32_t  *gain_cpx_p,
                               cfloat16_t  *scratch_p,
                               int32_t      phase_ramp,
                               int32_t      phase_init,
                               uint32_t     num_streams);
```

where:

inp_p – pointer to input/output buffer (16-bit complex Half-Precision, DMEM aligned)

gain_cpx_p – pointer to complex gain (32-bit complex Single-Precision, DMEM aligned)

scratch_p – pointer to scratch memory buffer (DMEM aligned)

phase_ramp – NCO base frequency (signed 32-bit integer, 1`s complement) corresponding to phase ramp

phase_init – NCO initial phase (signed 32-bit integer, 1's complement) corresponding to the initial phase of the NCO

num_streams – number of space-time streams (1 to 8)

Each kernel will perform the following computation in place:

$$x(k,m) = x(k,m) * g * \exp\left(-j * 2 * pi * \frac{f}{2^{32}} * (k + k_i)\right)$$

where:

$x(k,m)$ is the in/out subcarrier with sample index k and space-time stream m ("inp")

g is the complex gain ("gain_cpx")

f is the phase ramp ("phase_ramp")

k is the sample index (k = 0…N-1) where N is the total number of subcarriers per stream (64,128,..)

k_i is the initial phase ("phase_init") corresponding to the index of the 1st subcarrier

The header file provides several defines for the "phase_init" parameter depending on the use-case:

```
#define FREQ_DOMAIN_CORR_PHASE_INIT_11ac_80MHz_FULL          -122
#define FREQ_DOMAIN_CORR_PHASE_INIT_11ac_80MHz_QUART0        -122
#define FREQ_DOMAIN_CORR_PHASE_INIT_11ac_80MHz_QUART1         -58
#define FREQ_DOMAIN_CORR_PHASE_INIT_11ac_80MHz_QUART2           6
#define FREQ_DOMAIN_CORR_PHASE_INIT_11ac_80MHz_QUART3          70
```

Each space-time stream is contigously allocated in memory (no discontinuities within the same stream) and DMEM aligned but are not necessarily allocated one after the other. Subsequent streams are assumed allocated in memory at address offsets (expressed in half-words) equal to the following defines (for each kernel):

```
#define FREQ_DOMAIN_CORR_STREAM_OFF_64SBC      2*64
#define FREQ_DOMAIN_CORR_STREAM_OFF_128SBC     4*64
#define FREQ_DOMAIN_CORR_STREAM_OFF_256SBC     8*64
#define FREQ_DOMAIN_CORR_STREAM_OFF_512SBC    16*64
#define FREQ_DOMAIN_CORR_STREAM_OFF_1024SBC   32*64
```

NOTE: The above values correspond to the case when subsequent streams are allocated one ofter the other. The integrator can change the above values according to the actual DMEM stream allocation.

The format of the input/output space-time streams is depicted below for 11ac @ 80MHz full bandwidth:

S[-122][1]	S[-121][1]	S[-120][1]	S[-119][1]	…	S[-92][1]	S[-91][1]
…	…	…	…	…		
S[102][1]	S[103][1]	S[104][1]	S[105][1]	…	X	X
…	…	…	…	…	…	…
S[-122][2]	S[-121][2]	S[-120][2]	S[-119][2]	…	S[-92][2]	S[-91][2]
…	…	…	…	…		
S[102][2]	S[103][2]	S[104][2]	S[105][2]	…	X	X
…	…	…	…	…	…	…

where *S[k][m]* denotes the subcarrier with index *k* (*k* = -122 : 122) and space-time stream *m* (m = 1 … 8, each color above represents one stream). Each row above is a DMEM line, each containing 32 subcarriers. Each stream occupies 8 DMEM lines. Last DMEM line of each stream is not fully occupied due to the stream alignment ("don`t care" subcarriers are marked with "X"). The memory address increases linearly from left to right and from up to bottom. The address offset between two subsequent streams (the address offset between *S[-122][1]* and *S[-122][2]*) is assumed to be equal to the `FREQ_DOMAIN_CORR_STREAM_OFF_256SBC define (in half-words)`.

NOTE: Each input space-time stream is assumed to include data subcarriers, pilot subcarriers and also **NULL subcarriers** (corresponding to DC and its neighbours). Even though the NULL subcarriers are processed, their values should be ignored. Their presence in the input buffer is mandatory in order for the linear phase ramp correction to be properly applied in the entire bandwidth, without phase discontinuities.

DMEM requirements:

`freq_domain_corr_64sbc`

Variable	Size (half-words)	Minimum alignment (half-words)	Allocated by
`inp_p`	num_streams x 2 x 64	64	Caller
`gain_cpx_p`	4	4	Caller
`scratch_p`	2 x 64	64	Caller

`freq_domain_corr_128sbc`

Variable	Size (half-words)	Minimum alignment (half-words)	Allocated by
`inp_p`	num_streams x 4 x 64	64	Caller
`gain_cpx_p`	4	4	Caller
`scratch_p`	4 x 64	64	Caller

`freq_domain_corr_256sbc`

Variable	Size (half-words)	Minimum alignment (half-words)	Allocated by
`inp_p`	num_streams x 8 x 64	64	Caller
`gain_cpx_p`	4	4	Caller
`scratch_p`	8 x 64	64	Caller

`freq_domain_corr_512sbc`

Variable	Size (half-words)	Minimum alignment (half-words)	Allocated by
`inp_p`	num_streams x 16 x 64	64	Caller
`gain_cpx_p`	4	4	Caller
`scratch_p`	16 x 64	64	Caller

`freq_domain_corr_1024sbc`

Variable	Size (half-words)	Minimum alignment (half-words)	Allocated by
`inp_p`	num_streams x 32 x 64	64	Caller
`gain_cpx_p`	4	4	Caller
`scratch_p`	32 x 64	64	Caller

Test plan:

- Tested features:
  - 64, 128, 256, 512 subcarriers
  - 1 to 2 streams